

The Expected Wait for HHH

Part 1 — The Full Back-and-Forth, Round by Round

Three roads to one number, with every wrong equation, rebuttal, and recovery kept in

The problem. A fair coin is tossed repeatedly. Expected number of tosses until the first **HHH**? The answer is **14** — and it is the least interesting line in this document.

What this is. A faithful transcript of how the answer was *earned*, kept as a sequence of exchanges: each **Tried** (an actual attempt, wrong equations included), the **Caught** (why it breaks), and the **Clicked** (the recovery). The reasoning lives in the turns, not in the boxed result. The unifying thesis: the generating-function road and the Markov road are *the same act* — partition by an equivalence relation, then solve by the definition of expectation — differing only in *which axis* they cut along (ending time vs. first step).

1 Road 1 — Generating functions: partition by ending time

1.1 Round 1 — marginalizing without the constraint

Tried

Write the expectation as a weighted sum over the ending position and factor the probability:

$$E = \sum_{n \geq 3} n \mathbb{P}(\dots X_{n-2}=H, X_{n-1}=H, X_n=H), \quad \mathbb{P}(\dots) = \mathbb{P}(X_1 \dots X_{n-3}) \mathbb{P}(X_{n-2}=H, X_{n-1}=H, X_n=H).$$

“Isn’t the prefix $\mathbb{P}(X_1 \dots X_{n-3})$ the whole sample space, = 1?”

Caught

Two errors. (i) The event “ $X_{n-2}X_{n-1}X_n = \text{HHH}$ ” does *not* exclude HHH from having appeared earlier; but *first* occurrence at n requires “HHH ends at n and never before.” Dropping the second clause counts one sample path at many n — double counting. (ii) “Prefix is anything” as an *event* has probability 1, but any *specific* length- $(n-3)$ string has probability $(1/2)^{n-3}$. Conflating these is the tell: you are marginalizing when you must marginalize *under the constraint* “no HHH in the prefix.”

1.2 Round 2 — the single-point-mass survival, and the missing T

Tried

Patch in the constraint as

$$P_n = (1 - P_{n-3}) \cdot \frac{1}{8}.$$

“ $1 - P_{n-3}$ is the prefix having no HHH; $\frac{1}{8}$ is the last three being HHH.”

Caught

(i) $1 - P_{n-3}$ is wrong: P_{n-3} is the *single* probability that the first HHH ends exactly at $n-3$, so $1 - P_{n-3}$ means “first occurrence isn’t exactly at $n-3$ ” — not “no HHH in the

whole prefix.” Survival is one minus a *cumulative* sum, not minus one point mass. (ii) For HHH to end *first* at n , you also need $X_{n-3} = T$ (else $X_{n-3}X_{n-2}X_{n-1} = \text{HHH}$ ends it one step early); the definition is under-constrained. (iii) The $\frac{1}{8}$ assumes independence across a boundary that is actually coupled. **Numerical check:** this gives $P_4 = \frac{1}{8}$, but the true probability that the first HHH ends at toss 4 is $\frac{1}{16}$. Fails at $n = 4$.

1.3 Round 3 — the variable-length Q , and talking it out

Tried

Pin the last four as THHH and attach an extra factor Q for “no HHH in the rest”:

$$P_n = \frac{1}{16} \left(1 - \sum_{k=3}^{n-4} P_k Q \right), \quad \text{“} Q \text{ also says the remaining } n-4-k \text{ tosses have no HHH, else } P_k \text{ isn't enough.”}$$

Caught

Q is both redundant and double-counting: P_k already means *first* occurrence at k , so once it fires the path is already outside “no HHH in the prefix” — no extra factor belongs. And a single constant Q cannot express a *variable*-length gap $n-4-k$.

Pushed back

“ Q is obviously variable. But it still feels off — does P_k mean: in k tosses, HHH appears only in the last three?”

Clicked

That restatement is *correct* (first occurrence at $k \Leftrightarrow \text{HHH}$ only completes at the last three). The unease came from wanting P_k to *also* carry “after k , no HHH” — but P_k says nothing past position k . The fix is to stop trying to extend it: “HHH has occurred by toss m ” $= \sum_k P_k$ because the first-passage events are **mutually exclusive** (each path’s first hit time is unique), so survival is $1 - \sum_k P_k$, with *no* Q .

1.4 Round 4 — the overlap fear and the “not the whole space” unease

Pushed back

“But adding the P_k will overlap, won’t it? If P_3 (first three are HHH) is followed by another HHH in positions 4 to $n-4$, doesn’t that coincide with some later P_k ?” And later: “ P_k only governs the first k tosses — adding them up doesn’t feel like it covers the whole probability space.”

Caught — both worries are false alarms

Overlap: “first occurrence” makes each path’s hit time unique — a path lands in exactly one P_k . P_3 ’s paths already had their first HHH at position 3; a later HHH on the same path doesn’t put it into a later P_k . The events don’t overlap. *Whole space:* for survival you do *not* want $\sum_{k=3}^{n-4} P_k = 1$. That partial sum is “first HHH occurs within the first $n-4$ tosses,” and the leftover $1 - \sum$ is exactly the survival “no HHH yet” you need. “Doesn’t cover the whole space” is not a bug — it is the very piece the subtraction uses.

1.5 Round 5 — forcing it through brute enumeration

To trust nothing, list everything. $n = 4$ ($2^4 = 16$ strings): those containing HHH are HHHH, HHHT, THHH — 3 of them — so “no HHH in 4 tosses” = $\frac{13}{16}$. Classify by first-hit: P_3 (HHHH, HHHT) = $\frac{2}{16}$, P_4 (THHH) = $\frac{1}{16}$; sum $\frac{3}{16}$ matches the 3 strings. $n = 5$ (32 strings): 8 contain HHH, survival $\frac{24}{32} = \frac{3}{4}$; and $P_3 = \frac{4}{32}$, $P_4 = \frac{2}{32}$, $P_5 = \frac{2}{32}$ sum to $\frac{8}{32}$. Both check.

Pushed back

“That works *because* the pattern is HHH. For a different pattern it’d be shaky, no?”

Clicked — the framework is pattern-agnostic; only the boundary changes

Enumerate HT at $n = 4$: strings without HT are HHHH, THHH, TTHH, TTTH, TTTT — 5 of them — survival $\frac{5}{16}$, with $P_2 = \frac{4}{16}$, $P_3 = \frac{4}{16}$, $P_4 = \frac{3}{16}$, all consistent. The mutual-exclusivity framework never breaks. What *does* change per pattern is the boundary condition inside each P_k : HHH is **maximally self-overlapping** (its prefixes H, HH are also suffixes), so “first occurrence” becomes the clean “put a T one step back.” HT, with different overlap, becomes “the previous two aren’t HT.” Self-overlap is the hidden variable — and it is what drives the Penney’s-game non-transitivity in Part 2.

1.6 Round 6 — the generating function and the P_5 bug

The recursion is $P_n = \frac{1}{16}(1 - \sum_{k=3}^{n-4} P_k)$ for $n \geq 5$. Let $G(x) = \sum_{n \geq 3} P_n x^n$; multiplying by x^n , summing from $n = 5$, and swapping the double sum (inner survival sum $\rightarrow \frac{x^4}{1-x} G(x)$) gives

$$G(x) \cdot \frac{16 - 16x + x^4}{1 - x} = \frac{x^5}{1 - x} + 2x^3 + x^4.$$

Tried

Seed with hand-filled terms including $P_5 = \frac{1}{32}$, $P_6 = \frac{1}{16}$. Result: $E = G'(1) = 14.5$.

Caught

The algebra is fine and $G(1) = 1$ holds — but 14.5 is wrong (the classic answer is 14). The bug is the seed: P_5 was filled as $\frac{1}{32}$. Enumeration says $P_5 = \frac{1}{16}$ (the two paths HTHHH, TTHHH give $\frac{2}{32}$). Note $G(1) = 1$ stayed true *even with the bug* — normalization is insensitive to it — so only the *derivative* check exposes the missing 0.5.

Clicked

Don’t hand-fill P_5 at all. Seed only $P_3 = \frac{1}{8}$, $P_4 = \frac{1}{16}$ and let the recursion (valid for $n \geq 5$) generate the rest, verifying against enumeration at $n = 5, 6, 7$. The numerator then collapses:

$$G(x) = \frac{2x^3 - x^4}{16 - 16x + x^4} = \frac{x^3(2 - x)}{16 - 16x + x^4}, \quad E = G'(1) = \frac{2 \cdot 1 - 1 \cdot (-12)}{1} = 14.$$

Convergence note for the report: $G(x)$ converges for $|x| < 1$; the expectation is $G'(1)$ as the one-sided limit $x \rightarrow 1^-$, not a substitution $x = 1$. The series lives strictly inside the unit disk; we approach the boundary from the left.

2 Road 2 — The Markov chain, and the long detour through steady states

2.1 Building the machine, and the “how many tosses do we assume?” tangle

States = matched-suffix length of HHH: s_0, s_1, s_2 , plus absorbing HHH. Each transition 1/2:
 s_0 : $H \rightarrow s_1, T \rightarrow s_0$; s_1 : $H \rightarrow s_2, T \rightarrow s_0$; s_2 : $H \rightarrow \text{HHH}, T \rightarrow s_0$.

Tried

“Does a single toss put us into s_0 ? Do we have to assume how many tosses on average enter s_0 , then s_1, s_2 ?” (Repeatedly framed the per-state counts as something to *assume*.)

Caught

The logic is inverted: “how many tosses on average” is not an *input* you assume — it is the *output* you solve for. Once the transition probabilities are fixed (all 1/2), the expected counts are *locked*; there is no free parameter to set. Looking for “what to assume” is looking for a degree of freedom the fully-specified chain doesn’t have.

Clicked

“Set e_0, e_1, e_2 = expected tosses to absorption from each state.” Yes — that is setting *unknowns* (variables to be solved), not setting *assumptions* (given numbers). “Assume a value” was the wrong word; “name the unknown, then write its equations” is the right move.

2.2 The first wrong equation: three branches summing to 3/2

Tried

$$e_0 = \frac{e_0+1}{2} + \frac{e_1+1}{2} + \frac{e_2+1}{2}.$$

“Already tossed e_0 times, in state 0, toss once more, with prob 1/2 re-enter s_0 ...”

Caught

(i) Three branches each weighted 1/2 means s_0 goes to three places with prob 1/2 each — but s_0 has only *two* out-edges (T self-loop, H to s_1); there is *no* direct edge to s_2 . The weights sum to $3/2 \neq 1$ — not a probability distribution. (ii) “Already tossed e_0 times” is a semantic slip: e_0 is the *forward-looking expectation* from s_0 , not a count already spent. First-step analysis is “stand at s_0 , toss once more (count +1), land somewhere, add the expectation from there.”

2.3 “Has it become $Ax = x$?” — steady state vs. expectation

Tried

Strip the +1: $e_0 = \frac{e_0}{2} + \frac{e_1}{2} + \frac{e_2}{2}$. “Hasn’t this become the $Ax = x$ attractor equation?”

Caught

No. $Ax = x$ solves the steady-state *distribution* π , whose entries are *probabilities* summing to 1. The e_i are *expected steps* — units of “count,” not probabilities, not normalized. Matching the algebraic shape doesn’t make them the same equation. Worse, deleting the $+1$ destroys the soul of the expectation equation: that $+1$ is “count one toss per step,” the whole reason e_i measures steps. Strip it and the equation describes no waiting time at all. The steady-state equation isn’t obtained by “deleting a term” from the expectation equation; they have different origins (homogeneous $\pi = \pi A$ vs. inhomogeneous $e = 1 + \dots$, with that 1 the constant term).

2.4 The steady-state tangent, and where the useful information actually lives

Solving $\pi A = \pi$ with normalization gives the trivial $\pi = (0, 0, 0, 1)$: a legitimate fixed point ($A\pi = \pi$ holds; the absorbing row is $[0, 0, 0, 1]$) but *empty* — it says only “you end at HHH,” carrying no “how long.” Its apparent uniqueness is from having a single absorbing state, not from ergodicity. The expected wait is an *absorption time*, not a steady-state quantity.

Clicked — three connections that fell out here

(a) **Markov-ness is chosen, not given.** A single toss isn’t Markov; “matched-suffix length” lumps infinitely many histories into finite equivalence classes that *are* Markov — the same move that makes a bigram $p(w_t | w_{t-1})$ “Markov” only once the state carries the needed history. (b) **PageRank is the mirror image.** It *wants* the steady state, but the raw web graph isn’t ergodic (dangling nodes act absorbing); the teleport $A' = dA + (1-d)\frac{1}{N}\mathbf{1}\mathbf{1}^\top$ forces ergodicity so π is unique and meaningful. HHH *fears* the absorbing state (info in the transient); PageRank *destroys* absorbing behaviour (info in the steady state). (c) **The right tool for absorption time** is the fundamental matrix $N = (I - Q)^{-1}$ with Q the transient block; the i -th row sum is the expected steps from i . Steady state reads the eigenvector of full A (where mass ends); absorption time reads $(I - Q)^{-1}$ of Q (how long it lingers). Asking the former for the latter is why the steady-state road dead-ends.

2.5 The forward-equation disaster (the longest detour)

Tried

“You must reach j before reaching i — once you’re already at i , why look further?” So go forward, summing over *predecessors*:

$$E[T_i] = \sum_{\text{prev } j} \mathbb{P}(j \rightarrow i) (1 + E[T_j]).$$

Caught — with a counterexample

$\sum_j \mathbb{P}(j \rightarrow i)$ is a *column* sum of the transition matrix and is *not* 1 (row sums are 1), so $\mathbb{P}(j \rightarrow i)$ is not a valid weight. **Minimal counterexample:** by this formula $E_{s_1} = \mathbb{P}(s_0 \rightarrow s_1)(1 + E_{s_0}) = \frac{1}{2}(1 + 0) = \frac{1}{2}$. But the true expected time to first reach s_1 is the wait for the first head, geometric with $p = \frac{1}{2}$, = 2. Off by 4×. The ladder intuition (“must pass through j ”) is right, but “must pass through j ” $\neq$ “one step from j to i ”: you bounce on the s_0 self-loop many times; the formula counts only the single successful jump and drops the geometric wait.

Pushed back

“Now compute $\mathbb{P}(j \rightarrow i)$ from A — you need to column-orthogonalize, since $\sum_j \mathbb{P}(j \rightarrow i)$ should be 1.”

Caught — word and substance both

It is *normalization*, not orthogonalization (Gram–Schmidt has nothing to do with this). And the correct normalizing factor is not “divide by the column sum”: the true time-reversal is $\tilde{P}(i \rightarrow j) = \pi_j \mathbb{P}(j \rightarrow i) / \pi_i$, which reduces to dividing by the column sum *only when π is uniform*. Here $\pi = (0, 0, 0, 1)$ is degenerate ($\pi_0 = \pi_1 = \pi_2 = 0$ as denominators), so the reversal is undefined and the construction is invalid from the start.

Pushed back

“Just do the normalization — compute it.”

Caught

Column-normalizing A (each column $\div$ its sum: s_0, HHH have $\frac{3}{2}$; s_1, s_2 have $\frac{1}{2}$) gives

$$\tilde{A} = \begin{bmatrix} 1/3 & 1 & 0 & 0 \\ 1/3 & 0 & 1 & 0 \\ 1/3 & 0 & 0 & 1/3 \\ 0 & 0 & 0 & 2/3 \end{bmatrix}, \quad \text{row sums } (4/3, 4/3, 2/3, 2/3) \neq 1.$$

It is column-stochastic but the *rows* no longer sum to 1 — not a valid transition matrix — and it leaves a $2/3$ self-loop on HHH (a reverse trap). Mechanically constructible, but not a legitimate chain.

Pushed back

“Use my formula $E_i = \sum_j \tilde{A}[j \rightarrow i](1 + E_j)$, with $\tilde{A}[j \rightarrow i] = \text{row } j, \text{ column } i$. List the system.”

After aligning the row/column convention (collecting each state’s predecessors), the system is

$$e_0 = \frac{1}{3}(1 + e_0) + \frac{1}{3}(1 + e_1) + \frac{1}{3}(1 + e_2), \quad e_1 = 1 \cdot (1 + e_0), \quad e_2 = 1 \cdot (1 + e_1),$$

and solving:

$$e_1 = 1 + e_0, \quad e_2 = 2 + e_0, \quad e_0 = \frac{1}{3}[3 + e_0 + (1 + e_0) + (2 + e_0)] = 2 + e_0 \implies 0 = 2.$$

Caught — “why?!”

The system is inconsistent. The $0 = 2$ comes from e_0 appearing on *both* sides plus a positive constant: s_0 ’s predecessors include s_0 itself (the T self-loop), so the “arrival” equation says $e_0 = e_0 + (\text{positive})$ — “reaching s_0 from s_0 still costs positive steps,” a contradiction under the forward/arrival reading. Root causes, all three: the self-loop makes “arrival expectation” ill-defined on the start state; the $+1$ is the wrong increment for a variable geometric wait; and π is degenerate so the column-normalized $\tilde{A}$ isn’t a true reversal. The entire forward route is blocked for this chain.

Clicked — “maybe it’s +2” is the alarm bell

The decisive signal was the thought “maybe the increment should be +2 or +3, not +1.” That hesitation *is* the diagnosis: a constant +1 asserts “ $j \rightarrow i$ is exactly one step.” The moment it wants to be 2 or 3, you’ve discovered $j \rightarrow i$ is a *random geometric wait* (bouncing on the self-loop) that no fixed constant can encode — and patching with +2, +3 can’t fix a variable gap. This is exactly why the *backward* road is clean: its +1 counts “toss once, now,” always exactly one step.

3 Road 3 — Backward first-step analysis, derived not recited**3.1 “This feels like Bellman / DP”****Clicked**

Going backward from the terminal state *is* the Bellman equation, not merely “like” it. With e_i the value function, +1 the per-step cost, $\sum_j \mathbb{P}(i \rightarrow j)e_j = \mathbb{E}[V(\text{next})]$, $\gamma = 1$, and $e_{\text{HHH}} = 0$ the terminal boundary, $e_i = 1 + \sum_j \mathbb{P}(i \rightarrow j)e_j$ is the Bellman *expectation* equation (policy evaluation — no max, the coin is random and you don’t choose). Add a decision and a max wraps the sum, giving optimal stopping / an MDP. “Solve backward from the end” is backward induction.

3.2 Deriving the +1, and the notation that felt like a posterior**Tried**

Write the total-expectation split as $E[T_i] = \sum_j \mathbb{P}(i \rightarrow j) E[T_i \mid \text{first step to } j]$.

Caught — the wording

“First step” is the wrong frame when recursing by state — say “the next state is j ,” matching $\mathbb{P}(i \rightarrow j)$. Then: trying $E[T_i \mid i \rightarrow j]$ “feels like a posterior,” because the subscript i on T_i and the i in the condition $i \rightarrow j$ look like the same i twice — as if the start were both part of the variable and part of the condition.

Clicked — clean random variables dissolve it

Let T = steps still needed to absorption from wherever you are — *one* variable, no subscript. “From i ” is a condition: $e_i := \mathbb{E}[T \mid X_0 = i]$, with X_1 the next state. Now i lives only in the condition; T is the averaged object; the $A \mid B$ roles are unambiguous.

Total expectation (no Markov yet): $\{X_1 = j\}$ partition, so $\mathbb{E}[T \mid X_0 = i] = \sum_j \mathbb{P}(i \rightarrow j) \mathbb{E}[T \mid X_0 = i, X_1 = j]$.

Split the conditional: under $\{X_0 = i, X_1 = j\}$, $T = 1 + T'$ with 1 the first (certain) toss and T' the remainder from j . Linearity: $\mathbb{E}[T \mid X_0 = i, X_1 = j] = 1 + \mathbb{E}[T' \mid \dots]$. *Now* Markov: once at j , the future is independent of how you arrived, so $\mathbb{E}[T' \mid \dots] = \mathbb{E}[T \mid X_0 = j] = e_j$. Hence $\mathbb{E}[T \mid X_0 = i, X_1 = j] = 1 + e_j$, the +1 falling out of the split as the first toss — not bolted on. So

$$e_i = \sum_j \mathbb{P}(i \rightarrow j)(1 + e_j) = 1 + \sum_j \mathbb{P}(i \rightarrow j) e_j, \quad e_{\text{HHH}} = 0.$$

Clicked — the self-summary that names the whole method

“So it’s really: enumerate the partition (an equivalence relation), then set up and solve the equations.” Exactly. Two layers of partition stacked: first the no-after-effect equivalence relation that lumps infinitely many histories into finite states (this is what makes the unknowns finite and the system solvable), then the total-expectation partition by next state. First-step analysis = Bellman expectation = backward DP, all one act.

3.3 Solving, and the cross-check

$$e_0 = 1 + \frac{1}{2}e_0 + \frac{1}{2}e_1, \quad e_1 = 1 + \frac{1}{2}e_0 + \frac{1}{2}e_2, \quad e_2 = 1 + \frac{1}{2}e_0.$$

From the first, $e_0 = 2 + e_1$; substituting $e_2 = 1 + \frac{1}{2}e_0$ into the second gives $e_1 = \frac{3}{2} + \frac{3}{4}e_0$; with $e_1 = e_0 - 2$, $\frac{1}{4}e_0 = \frac{7}{2}$, so

$$e_0 = 14, \quad e_1 = 12, \quad e_2 = 8.$$

$e_0 = 14$ matches $G'(1) = 14$ from Road 1 — two independent methods cross-validated. ($14 > 12 > 8$ is sensible: further along the matched suffix means closer to HHH, smaller remaining wait.)

4 The unifying picture: one act, two partition axes

Strip both successful roads to the skeleton and they are identical: **find a no-after-effect equivalence relation that collapses infinitely many cases into finitely many, then solve by the linearity of expectation.** They differ only in the axis of partition.

Road 1 (generating function) — partition by ENDING. Cut by “first HHH ends exactly at toss n ,” giving mutually exclusive $\{P_n\}$; unfold along the *time* axis; package the infinitely many P_n into $G(x)$; solve by $E = \sum_n nP_n = G'(1)$. Partition variable: *when* it ends.

Road 2/3 (Markov / first-step) — partition by FIRST STEP / current state. Cut by “matched-suffix length” (s_0, s_1, s_2) , folding infinitely many histories into finite classes; unfold along the *state* axis; solve by $e_i = 1 + \sum_j \mathbb{P}(i \rightarrow j)e_j$. Partition variable: *where* the first step goes.

Same recursion, two slices. One spreads the structure along time (infinitely many P_n , packaged by a generating function); the other folds it along states (finitely many e_i). Both use only an equivalence partition plus the definition of expectation. They *must* agree on 14 — and that agreement is the cross-check that neither road hides an error.

5 What actually transfers

- **First occurrence $\Leftrightarrow$ mutual exclusivity:** $\{P_k\}$ partition; survival $= 1 - \sum_k P_k$; no Q . The overlap fear and the “not the whole space” unease are both false alarms.
- **Always sanity-check by enumeration** ($n = 4, 5$; HT) before trusting algebra. And test $G'(1)$, not just $G(1)$: normalization hides the P_5 bug; the derivative exposes it.
- **Self-overlap is the hidden variable:** it sets each pattern’s boundary condition and drives Penney’s-game non-transitivity (Part 2).
- **Markov-ness is chosen, not given;** steady state (A ’s eigenvector) $\neq$ absorption time $((I - Q)^{-1}$ of Q); PageRank is the ergodic mirror (teleport).

- **Forward fails, backward is clean:** “maybe it’s +2” is the alarm you’re on the forward road; the backward +1 is “toss once now,” always one step.
- **First-step analysis = Bellman expectation = backward DP;** derive +1 from $T = 1 + T'$; clean notation $\mathbb{E}[T \mid X_0 = i]$, never $\mathbb{E}[T_i \mid i \rightarrow j]$.
- **Generating function and Markov are the same act,** partitioned by ending time vs. first step. Cross-validating them is free insurance.

Part 2: expected waits across patterns (HHH vs. HTH and friends), where self-overlap stops being a footnote and produces the counterintuitive non-transitivity of Penney’s game.